Program 1 - Get the Largest Number from a List

Step 1: Start
Step 2: Initialise an empty list list1.
Step 3: Read an integer and append it to list1.
Step 4: Read whether it is the end of the list (y/n) into flag.
Step 5: If flag is not 'y', go to Step 3.
Step 6: Display list1.
Step 7: Find the maximum value in list1 using max() and display it.
Step 8: Stop

Program 2 - Remove Duplicates from a List

Step 1: Start
Step 2: Initialise list1 with duplicate elements and display it.
Step 3: Initialise an empty list list2.
Step 4: For each element i in list1, if i is not in list2, append i to list2.
Step 5: Display list2 as the deduplicated list.
Step 6: Stop

Program 3 - Conversion of Tuple to Dictionary

Step 1: Start
Step 2: Create a tuple of (name, mark) pairs and display it.
Step 3: Convert the tuple into a dictionary using dict().
Step 4: Display the dictionary.
Step 5: Stop

Program 4 - Merging Dictionaries

Step 1: Start
Step 2: Create dictionaries dict1 and dict2 and display them.
Step 3: Merge dict2 into dict1 using the update() method.
Step 4: Display the merged dictionary dict1.
Step 5: Stop

Program 5 - Checking Common Member in Lists

Step 1: Start
Step 2: Initialise list1 and convert it into a set set1.
Step 3: Initialise list2 and convert it into a set set2.
Step 4: Find the common elements as result = set1 & set2.
Step 5: If result is an empty set, display False, otherwise display True.
Step 6: Stop

Program 6 - Find Earlier Date

Step 1: Start
Step 2: Initialise date1 and date2 as strings in YYYY-MM-DD format.
Step 3: If date1 < date2, display date1 as the earlier date.
Step 4: Else if date1 > date2, display date2 as the earlier date.
Step 5: Otherwise display that the two dates are equal.
Step 6: Stop

Program 7 - Subtract 5 Days from Current Date

Step 1: Start
Step 2: Import the datetime module.
Step 3: Get the current date using datetime.date.today().
Step 4: Create a timedelta object of 5 days.
Step 5: Subtract the timedelta from the current date and display the result.
Step 6: Stop

Program 8 - File Copy Operation

Step 1: Start
Step 2: Open file1.txt in read mode.
Step 3: Read the entire contents of the file and close it.
Step 4: Open file2.txt in write mode.
Step 5: Write the contents into file2.txt and close it.
Step 6: Stop

Program 9 - Capitalise Each Word in a File

Step 1: Start
Step 2: Open file1.txt in read mode.
Step 3: Read the entire contents and convert them to uppercase using upper().
Step 4: Close the file.
Step 5: Open file1.txt in write mode.
Step 6: Write the new contents into the file and close it.
Step 7: Stop

Program 10 - Search and Replace in a File

Step 1: Start
Step 2: Open file1.txt in read mode.
Step 3: Read the entire contents and replace the word 'Python' with 'C' using replace().
Step 4: Close the file.
Step 5: Open file1.txt in write mode.
Step 6: Write the new contents into the file and close it.
Step 7: Stop

Program 11 - Count Number of Lines in a File

Step 1: Start
Step 2: Initialise count = 0.
Step 3: Open file1.txt in read mode.
Step 4: For each line in the file, increment count by 1.
Step 5: Display count as the number of lines.
Step 6: Stop

Program 12 - Retrieve Lines Having Two Consecutive 1's

Step 1: Start
Step 2: Initialise an empty list lines_with_consecutive_ones.
Step 3: Open file1.txt in read mode.
Step 4: For each line in the file, if '11' is present in the line, strip the line and append it to the list.
Step 5: Display the list.
Step 6: Display each line in the list.
Step 7: Stop

Program 13 - Matrix Operations Using Vectorisation

Step 1: Start
Step 2: Import numpy.
Step 3: Create two 3 x 3 matrices, matrix1 and matrix2, and display them.
Step 4: Compute the sum using numpy.add().
Step 5: Compute the difference using numpy.subtract().
Step 6: Compute the product using numpy.matmul().
Step 7: Compute the transpose of matrix1 using numpy.transpose().
Step 8: Compute the scalar multiple 2 * matrix1.
Step 9: Display all the results.
Step 10: Stop

Program 14 - Creation of Matrices for 2D Geometric Transformations

Step 1: Start
Step 2: Import numpy.
Step 3: Define translationMatrix(tx, ty) that returns the 3 x 3 matrix [[1,0,tx],[0,1,ty],[0,0,1]].
Step 4: Define rotationMatrix(degree): convert degree to radians theta, find c = cos(theta) and s = sin(theta), and return [[c,-s,0],[s,c,0],[0,0,1]].
Step 5: Define scalingMatrix(sx, sy) that returns [[sx,0,0],[0,sy,0],[0,0,1]].
Step 6: Call each function with sample values and display the matrices.
Step 7: Stop

Program 15 - Creation of Matrices for 3D Geometric Transformations

Step 1: Start
Step 2: Import numpy.
Step 3: Define translationMatrix(tx, ty, tz) that returns the 4 x 4 matrix with tx, ty, tz in the last column and 1 on the diagonal.
Step 4: Define the rotation matrix about the x axis: convert degree to radians, find c and s, and return the 4 x 4 matrix in which the y-z block is [[c,-s],[s,c]].
Step 5: Define the rotation matrix about the y axis: the x-z block is [[c,s],[-s,c]].
Step 6: Define the rotation matrix about the z axis: the x-y block is [[c,-s],[s,c]].
Step 7: Define scalingMatrix(sx, sy, sz) with sx, sy, sz on the diagonal and 1 as the last diagonal element.
Step 8: Call each function with sample values and display the matrices.
Step 9: Stop

Program 16 - Singular Value Decomposition of a Matrix

Step 1: Start
Step 2: Import matrix, matmul and diag from numpy and svd from scipy.linalg.
Step 3: Define a 3 x 3 matrix A and display it.
Step 4: Decompose A into U, S and V using svd(A), where S holds the singular values.
Step 5: Display U, S and V.
Step 6: Create the diagonal matrix Sigma from S using diag().
Step 7: Reconstruct the matrix as B = U x Sigma x V using matmul().
Step 8: Display B.
Step 9: Stop

Program 17 - Plot a Histogram

Step 1: Start
Step 2: Import pyplot from matplotlib and array from numpy.
Step 3: Create a figure and axes using pyplot.subplots(1,1).
Step 4: Store the marks in an array.
Step 5: Draw the histogram using hist() with bins 0, 10, 20, ..., 100.
Step 6: Set the title, x ticks, x label and y label.
Step 7: Display the plot using pyplot.show().
Step 8: Stop

Program 18 - Plot a Histogram Using Iris Data Set

Step 1: Start
Step 2: Import pyplot from matplotlib and import pandas.
Step 3: Read iris.csv into a data frame df using read_csv().
Step 4: Create a figure and axes using pyplot.subplots(1,1).
Step 5: Plot the histogram of the petal.length column using plot(kind='hist').
Step 6: Set the title, x ticks, y ticks and x label.
Step 7: Display the plot using pyplot.show().
Step 8: Stop

Program 19 - Plot a Scatterplot

Step 1: Start
Step 2: Import pyplot from matplotlib.
Step 3: Store the roll numbers and marks in two lists.
Step 4: Create a figure and axes using pyplot.subplots(1,1).
Step 5: Draw the scatter plot of rollnos (x axis) against marks (y axis) using scatter().
Step 6: Set the title, ticks and axis labels.
Step 7: Display the plot using pyplot.show().
Step 8: Stop

Program 20 - Plot a Scatter plot Using Iris Data Set

Step 1: Start
Step 2: Import pyplot from matplotlib and import pandas.
Step 3: Read iris.csv into a data frame df.
Step 4: Create a figure and axes and a dictionary of colours for each species.
Step 5: Group the data frame by the species column.
Step 6: For each group, draw a scatter plot of petal.length against petal.width in the colour of that species.
Step 7: Set the title and axis labels.
Step 8: Display the plot using pyplot.show().
Step 9: Stop

Program 21 - Predict the Food Type Using k-NN algorithm

Step 1: Start
Step 2: Read food.csv into a data frame food.
Step 3: Create a data frame tomato with sweetness = 6 and crunchiness = 4.
Step 4: Select the sweetness and crunchiness columns of food and tomato as the training and test features.
Step 5: Read the value of k from the user and load the class package.
Step 6: Apply knn(): find the Euclidean distance from tomato to every food item, select the k nearest items and choose the most frequent FoodType among them.
Step 7: Display the predicted food type.
Step 8: Stop

Program 22 - Classify a Student Using k-NN algorithm

Step 1: Start
Step 2: Load the class package and read survey.csv into a data frame survey.
Step 3: Create a data frame student with X = 5 and Y = 7.
Step 4: Select the first two columns (X, Y) of survey as the training features.
Step 5: Apply knn() with k = 3 using Z as the class label.
Step 6: Display the predicted classification of the student.
Step 7: Stop

Program 23 - Prediction of Cancer Using k-NN algorithm

Step 1: Start
Step 2: Load the class and gmodels packages and read wisc_bc_data.csv into wbcd.
Step 3: Define normalize(x) = (x - min(x)) / (max(x) - min(x)).
Step 4: Apply normalize to columns 3 to 31 of wbcd to get wbcd_n.
Step 5: Take rows 1 to 469 as training data and rows 470 to 569 as test data.
Step 6: Take the diagnosis column (column 2) of the same rows as training and test labels.
Step 7: Predict the test labels using knn() with k = 21 and display them.
Step 8: Compare the predicted and actual labels using CrossTable().
Step 9: Stop

Program 24 - Prediction of Flu Using Naive Bayes Algorithm

Step 1: Start
Step 2: Load the e1071 package and read symptoms.csv into a data frame.
Step 3: Take columns 1 to 4 as training data and HasFlu as the class label.
Step 4: Create the test data frame with Chills = Y, RunningNose = N, Headache = mild and Fever = Y.
Step 5: Build the model using naiveBayes(), which computes the prior and conditional probabilities.
Step 6: Display the model.
Step 7: Predict the class of the test data using predict() and display it.
Step 8: Stop

Program 25 - Prediction of Stealing Using Naive Bayes Algorithm

Step 1: Start
Step 2: Load the e1071 package and read cars.csv into a data frame.
Step 3: Take columns 2 to 4 (Colour, Type, Origin) as training data and column 5 as the class label.
Step 4: Create the test data frame with Colour = red, Type = SUV and Origin = domestic.
Step 5: Build the model using naiveBayes() and display it.
Step 6: Predict the stolen status of the test data using predict() and display it.
Step 7: Stop

Program 26 - Prediction of Iris Species Using Naive Bayes Algorithm

Step 1: Start
Step 2: Load the e1071, caTools and gmodels packages and read iris.csv.
Step 3: Set the seed using set.seed(100).
Step 4: Split the data in the ratio 70:30 using sample.split() on the species column.
Step 5: Create the training set (split == TRUE) and the test set (split == FALSE).
Step 6: Take columns 1 to 4 as features and column 5 as the class label for both sets.
Step 7: Build the model using naiveBayes() on the training data and display it.
Step 8: Predict the species of the test data and display the result.
Step 9: Evaluate the prediction using CrossTable().
Step 10: Stop

Program 27 - Prediction of Cheating Using C5.0 Decision Tree Algorithm

Step 1: Start
Step 2: Load the C50 and gmodels packages and read person.csv into a data frame.
Step 3: Take rows 1 to 8 as training data and rows 9 to 10 as test data, leaving out column 5 (Cheats?).
Step 4: Take column 5 of the same rows as training and test labels.
Step 5: Build the decision tree using C5.0() with the training data and the labels converted to a factor.
Step 6: Display the model and its summary.
Step 7: Predict the labels of the test data using predict() and display them.
Step 8: Evaluate the prediction using CrossTable().
Step 9: Stop

Program 28 - Prediction of Cricket Play Using C5.0 Decision Tree Algorithm

Step 1: Start
Step 2: Load the C50 and gmodels packages and read cricket.csv into a data frame.
Step 3: Take rows 1 to 9 as training data and rows 10 to 14 as test data, leaving out column 5 (Play Cricket).
Step 4: Take column 5 of the same rows as training and test labels.
Step 5: Build the decision tree using C5.0() with the training data and the labels converted to a factor.
Step 6: Display the model and its summary.
Step 7: Predict the labels of the test data using predict() and display them.
Step 8: Evaluate the prediction using CrossTable().
Step 9: Stop

Program 29 - Identification of Risky Bank Loans Using C5.0 Decision Tree Algorithm

Step 1: Start
Step 2: Load the C50 and gmodels packages and read credit.csv into a data frame.
Step 3: Take rows 1 to 900 as training data and rows 901 to 1000 as test data, leaving out column 17 (default).
Step 4: Take column 17 of the same rows as training and test labels.
Step 5: Build the decision tree using C5.0() with the training data and the labels converted to a factor.
Step 6: Display the model and its summary.
Step 7: Predict the labels of the test data using predict() and display them.
Step 8: Evaluate the prediction using CrossTable().
Step 9: Stop

Program 30 - Prediction of Stock Index Price Using Multiple Linear Regression Technique

Step 1: Start
Step 2: Read economy.csv into a data frame.
Step 3: Take rows 1 to 18 as training data and rows 19 to 24 as test data.
Step 4: Build the multiple linear regression model using lm(), with Stock_Index_Price as the dependent variable and Interest_Rate and Unemployment_Rate as the independent variables.
Step 5: Display the estimated regression coefficients.
Step 6: Evaluate the model using summary() (R-squared, p-values).
Step 7: Predict the values for the test data using predict().
Step 8: Display the predictions.
Step 9: Stop

Program 31 - Prediction of Heart Disease Using Multiple Linear Regression Technique

Step 1: Start
Step 2: Read heart.csv into a data frame.
Step 3: Take rows 1 to 400 as training data and rows 401 to 498 as test data.
Step 4: Build the multiple linear regression model using lm(), with heart.disease as the dependent variable and biking and smoking as the independent variables.
Step 5: Display the estimated regression coefficients.
Step 6: Evaluate the model using summary() (R-squared, p-values).
Step 7: Predict the values for the test data using predict().
Step 8: Display the predictions.
Step 9: Stop

Program 32 - Prediction of Medical Expenses Using Multiple Linear Regression Technique

Step 1: Start
Step 2: Read insurance.csv into a data frame.
Step 3: Take rows 1 to 1000 as training data and rows 1001 to 1338 as test data.
Step 4: Build the multiple linear regression model using lm(), with expenses as the dependent variable and age, sex, bmi, children, smoker and region as the independent variables.
Step 5: Display the estimated regression coefficients.
Step 6: Evaluate the model using summary() (R-squared, p-values).
Step 7: Predict the values for the test data using predict().
Step 8: Display the predictions.
Step 9: Stop

Program 33 - Partition of Iris Data Set Using k-means Clustering Algorithm

Step 1: Start
Step 2: Set the seed using set.seed(100).
Step 3: Read iris.csv into a data frame iris and make a copy iris2.
Step 4: Remove the species class label from iris2.
Step 5: Perform k-means clustering on iris2 using kmeans() with k = 3.
Step 6: Display the cluster object (sizes, means, assignments).
Step 7: Compare the clusters with the species labels using table().
Step 8: Stop

Program 34 - Partition of Breast Cancer Data Set Using k-means Clustering Algorithm

Step 1: Start
Step 2: Set the seed using set.seed(100).
Step 3: Read wisc_bc_data.csv into a data frame and make a copy.
Step 4: Remove the id and diagnosis columns from the copy.
Step 5: Perform k-means clustering on the copy using kmeans() with k = 2.
Step 6: Display the cluster object (sizes, means, assignments).
Step 7: Compare the clusters with the diagnosis labels using table().
Step 8: Stop